```
6.095s
6.1s
6.107s
6.12s
6.12s
6.127s

[info] Run completed in 7 seconds, 187 milliseconds.
```

Each line of output is printed almost simultaneously. The total duration of each call, about 6 seconds, is much longer than before. But all the calls started almost simultaneously, which makes a big difference. On the other hand, they took more time to compute because each call has only one thread to run `parseDoc`. However, overall performance is much better now that they run in parallel.

Since this laptop has hyperthreading, calculations can run up to 12 processes in parallel. *Hyperthreading* here means the computer can use each of its six real cores like two virtual cores. That results in 12 effective cores. But what if you have 100 elements instead of 10? Can you increase the number of threads to 100? You can, but you won't get anywhere near a linear increase in performance. You can effectively run only 12 processes in parallel, and if you have more threads, you pay the price of context switching.

16.1.2 Non-CPU bounded services

If the function you are calling is running on another machine, you can increase the parallelism much more because you are essentially waiting for the result. To simulate this scenario, the call to the external process is implemented by a scheduler that waits 1 second before returning the result. The machine on which you run the test is idle in the meantime. To ensure that the stream has the expected performance, you can specify a few seconds for the result in